



Universidade Federal de Sergipe
Centro de Ciências Exatas e Tecnologia
Departamento de Ciência da Computação



Disciplina: PROJETO E ANÁLISE DE ALGORITMOS (PROCC0083)

Período: 2026.2

Turma: T01

Prof(a): GLAUCO DE FIGUEIREDO CARNEIRO

Equipe: 03

Relatório da AV1

1. Identificação da equipe

Amintas Gomes de Moraes Neto
Ântones do Nascimento Lima
Flavio Emanuel de Oliveira Santos
Gabriel Gomes Attie
Ismael Teles Santos
João Fernandes Santos Filho

2. Tema, corpus e fonte

Tema: 7 — Documentação de APIs públicas

Corpus:

https://github.com/Khozex/PAA_UFS_2026_2_Neto_Amintas_Lima_Antones_Attie_Gabriel_Santos_Flavio_Teles_Ismael_Fernandes_Joao/blob/main/artefatos/ficha-do-corpus.pdf

Fonte: Github REST API v3. Disponível em: <https://docs.github.com/en/rest?apiVersion=2026-03-10>.
Acesso em: 24 de setembro de 2026.

3. URL do repositório, licença, commit/tag/release e data de acesso

Código-fonte:

https://github.com/Khozex/PAA_UFS_2026_2_Neto_Amintas_Lima_Antones_Attie_Gabriel_Santos_Flavio_Teles_Ismael_Fernandes_Joao

Licença: a definir

Release: 1.0

Data de acesso: 24/09/2026

4. Definição formal do problema

O desenvolvimento de aplicações modernas depende amplamente do consumo de APIs, tornando a documentação técnica um recurso fundamental para que desenvolvedores possam identificar e utilizar corretamente os serviços e operações disponibilizados por essas interfaces. Entretanto, documentações extensas de APIs públicas podem apresentar grande quantidade de informações, distribuídas entre

diferentes categorias, recursos, métodos HTTP, parâmetros e regras de utilização, dificultando a localização manual do *endpoint* adequado para atender a uma determinada necessidade expressa em linguagem natural.

Nesse contexto, este trabalho propõe investigar o problema de recuperação de informação em documentação de APIs públicas, tendo como corpus a documentação da GitHub REST API, na versão de 10 de março de 2026. O problema pode ser formalizado da seguinte maneira: dada uma consulta em linguagem natural realizada por um desenvolvedor, identificar e recuperar o *endpoint* da API que melhor corresponde à operação solicitada.

Por exemplo, consultas como “Como criar um repositório no GitHub via API?”, “Como criar uma issue em um repositório?” ou “Como criar uma nova branch?” exigem que o sistema seja capaz de relacionar a intenção descrita pelo usuário às informações técnicas presentes na documentação, retornando a operação correspondente. Dessa forma, o problema não consiste apenas em localizar termos semelhantes, mas em estabelecer uma relação entre a intenção da consulta e a funcionalidade efetivamente oferecida por determinado *endpoint*.

A escolha da GitHub REST API versão 3.0 como corpus é justificada por sua abrangência, diversidade e estruturação formal. A especificação OpenAPI disponibilizada oficialmente pelo GitHub permite representar cada operação de forma estruturada, contendo informações como método HTTP, caminho, parâmetros, descrição e *operationId*. Essa estrutura possibilita estabelecer uma unidade documental bem definida — o *endpoint* — e, principalmente, um identificador único para cada operação. Consequentemente, torna-se possível construir um gabarito objetivo de relevância para avaliar os resultados dos métodos de recuperação empregados.

O corpus selecionado apresenta aproximadamente 1.224 operações, distribuídas em 47 categorias e 140 subcategorias, totalizando cerca de 75 mil palavras de descrição. Essa dimensão proporciona um cenário suficientemente diversificado para avaliar a capacidade de diferentes estratégias de recuperação em distinguir operações semanticamente próximas, especialmente quando diferentes *endpoints* pertencem ao mesmo recurso ou apresentam descrições com vocabulário semelhante.

Assim, a investigação busca avaliar em que medida técnicas de recuperação de informação são capazes de mapear consultas formuladas em linguagem natural para operações técnicas específicas de uma API, utilizando uma fonte estruturada e reproduzível. A existência de uma correspondência objetiva entre consulta, *endpoint* e *operationId* também permite realizar uma avaliação quantitativa dos métodos utilizados, contribuindo para a análise da eficácia das estratégias de indexação, segmentação e recuperação aplicadas ao corpus.

5. Representação dos dados e estratégia de chunking

Não utilizada. Cada documento é a operação completa. Os diferentes tamanhos de corpus exigidos pelo experimento são obtidos por subconjuntos dos N primeiros documentos do docs.jsonl, o que varia N sem alterar a unidade de recuperação nem os pesos por campo.

6. Algoritmos e pseudocódigo

6.1 Estratégias de ordenamento

6.1.1 Quicksort

Algoritmo Quicksort com particionamento de lomuto.

Pseudocódigo:

```
QUICKSORT(A, lo, hi, less)
1  se lo < hi
2      p = PARTITION(A, lo, hi, less)
3      QUICKSORT(A, lo, p-1, less)
4      QUICKSORT(A, p+1, hi, less)
```

```
PARTITION(A, lo, hi, less)
1  pivo = A[hi]
2  i = lo - 1
3  para j = lo até hi-1
4      se less(A[j], pivo)
5          i = i + 1
6          troca A[i], A[j]
7  troca A[i+1], A[hi]
8  retorna i+1
```

6.1.2 Heapsort

Algoritmo Heapsort com sift-down (descida) ou heapify.

Pseudocódigo:

```
SIFT-DOWN(A, raiz, fim, less)
1  enquanto verdadeiro
2      filho = 2*raiz + 1
3      se filho > fim: retorna
4      se filho+1 <= fim e less(A[filho], A[filho+1])
5          filho = filho + 1
6      se less(A[raiz], A[filho])
7          troca A[raiz], A[filho]
```

```

8         raiz = filho
9     senão
10        retorna

BUILD-MAX-HEAP(A, n, less)
1  para i = piso(n/2)-1 até 0, decrescendo
2      SIFT-DOWN(A, i, n-1, less)

HEAPSORT(A, n, less)
1  BUILD-MAX-HEAP(A, n, less)
2  para fim = n-1 até 1, decrescendo
3      troca A[0], A[fim]
4      SIFT-DOWN(A, 0, fim-1, less)

```

6.1.3 MergeSort

Algoritmo Mergesort:

```

MERGE-SORT(A, lo, hi, less)
1  se hi - lo <= 1: retorna
2  meio = (lo + hi) / 2
3  MERGE-SORT(A, lo, meio, less)
4  MERGE-SORT(A, meio, hi, less)
5  MERGE(A, lo, meio, hi, less)

MERGE(A, lo, meio, hi, less)    // usa buffer auxiliar B do tamanho
de A
1  i = lo; j = meio; k = lo
2  enquanto i < meio e j < hi
3      se less(A[i], A[j])
4          B[k] = A[i]; i = i+1
5      senão
6          B[k] = A[j]; j = j+1
7      k = k+1
8  copia o restante de A[i..meio-1] (se houver) para B
9  copia o restante de A[j..hi-1] (se houver) para B
10 copia B[lo..hi-1] de volta para A[lo..hi-1]

```

6.2 Estratégias de Busca

6.2.1 Busca Indexada

Algoritmo de busca indexada dos k resultados mais relevantes

```

ALGORITMO BUSCA_INDEXADA(DOCUMENTOS, ÍNDICES, CONSULTA, K)

```

```

1 TERMOS ← NORMALIZAR(CONSULTA)
2
3 SE tamanho(TERMOS) = 0 ENTÃO
4     RETORNE ∅
5 FIM-SE
6
7 CANDIDATOS ← ∅
8 PARA i ← 1 ATÉ tamanho(TERMOS) FAÇA
9     // Lista de índices de documentos em que o termo aparece
10    POSTINGS ← ÍNDICES[TERMO]
11    PARA j ← 1 ATÉ tamanho(POSTINGS) FAÇA
12        DOC_ID ← POSTINGS[j]
13        // Por ser um hashmap "candidatos" ignora a inserção de
valores repetidos
14        INSIRA DOC_ID EM CANDIDATOS
15    FIM-PARA
16 FIM-PARA
17 RESULTADOS ← ∅
18 PARA CADA DOC_ID EM CANDIDATOS FAÇA
19     DOC ← DOCUMENTOS[DOC_ID]
20     SCORE ← CALCULARSCORE(CORPUS, DOC, TERMOS)
21     SE score = 0 ENTÃO
22         CONTINUE
23     FIM-SE
24     HIT ← (DOC, SCORE)
25     RESULTADOS ← INSERT_TOP_K(TOP, HIT, K)
26 FIM-PARA
27 RETORNE RESULTADOS
28 FIM-ALGORITMO

```

Algoritmo de normalização do texto da consulta

```

ALGORITMO NORMALIZAR(CONSULTA)
1 TOKENS ← vetor vazio
2 CONSULTA ← LOWERCASE(CONSULTA)
3 PALAVRAS ← TOKENIZE(CONSULTA)
4
5 i ← 1
6 ENQUANTO i ≤ tamanho(PALAVRAS) FAÇA
7     TOKEN ← PALAVRAS[i]
8     // STOPWORDS é um conjunto de palavras a serem ignoradas
9     EXISTE ← STOPWORDS[TOKEN]

```

```

9     SE EXISTE = FALSO ENTÃO
10         INSIRA TOKEN NO FINAL DE TOKENS
11     FIM-SE
12      $i \leftarrow i + 1$ 
13 FIM-ENQUANTO
14
15 RETORNE TOKENS
16 FIM-ALGORITMO

```

Algoritmo de inserção dos k resultados mais relevantes

```

ALGORITMO INSERT_TOP_K(RESULTADOS, HIT, K)
1   $n \leftarrow \text{tamanho}(\text{RESULTADOS})$ 
2  SE  $n = K$  ENTÃO
3      SE BETTER(HIT, TOP[n]) = FALSO ENTÃO
4          RETORNE RESULTADOS
5      FIM-SE
6       $n \leftarrow n - 1$ 
7  FIM-SE
8
9  RESULTADOS[n + 1]  $\leftarrow$  HIT
10  $n \leftarrow n + 1$ 
11 POS  $\leftarrow$  n
12
13 ENQUANTO POS > 1 E BETTER(RESULTADOS[POS], TOP[POS - 1]) =
VERDADEIRO FAÇA
14     AUX  $\leftarrow$  RESULTADOS[POS]
15     RESULTADOS[POS]  $\leftarrow$  RESULTADOS[POS - 1]
16     RESULTADOS[POS - 1]  $\leftarrow$  AUX
17     POS  $\leftarrow$  POS - 1
18 FIM-ENQUANTO
19
20 RETORNE RESULTADOS
21 FIM-ALGORITMO

```

Algoritmo de comparação

```

ALGORITMO BETTER(A, B)
1  SE A.Score  $\neq$  B.Score ENTÃO
2      SE A.Score > B.Score ENTÃO
3          RETORNE VERDADEIRO
4      SENÃO
5          RETORNE FALSO
6  FIM-SE

```

```

7 FIM-SE
8
9 SE A.TamanhoDocumento ≠ B.TamanhoDocumento ENTÃO
10     SE A.TamanhoDocumento > B.TamanhoDocumento ENTÃO
11         RETORNE VERDADEIRO
12     SENÃO
13         RETORNE FALSO
14 FIM-SE
15 FIM-SE
16
17 SE A.Doc.ID < B.Doc.ID ENTÃO
18     RETORNE VERDADEIRO
19 SENÃO
20     RETORNE FALSO
21 FIM-SE
22 FIM-ALGORITMO

```

7. Justificativa de corretude

7.1 Estratégias de ordenamento

7.1.1 Quicksort

PARTITION (Invariante de laço): no início de cada iteração j , para todo índice x em $[lo, i]$, $A[x]$ vem antes do pivô; para todo x em $[i+1, j-1]$, $A[x]$ não vem antes do pivô; $A[hi] = \text{pivo}$.

Inicialmente, $i = lo-1$, ambas as faixas $[lo, i]$ e $[i+1, j-1]$ são vazias.

Progressivamente, se $A[j]$ vem antes do pivô, incrementa i e troca $A[i]$ com $A[j]$, movendo o elemento pequeno para o fim da faixa "menores"; caso contrário, j avança sem alterar as faixas — o invariante se mantém em ambos os casos.

Finalmente, quando $j = hi$, todo o intervalo $[lo, hi-1]$ foi particionado; a troca final (linha 7) coloca o pivô entre as duas faixas, produzindo $A[lo..i] < \text{pivo} \leq A[i+1..hi]$. **PARTITION** retorna a posição final do pivô.

QUICKSORT (Corretude):

Caso base: $lo \geq hi$ — subarray com 0 ou 1 elemento, trivialmente ordenado.

Passo indutivo: assume-se que **QUICKSORT** ordena corretamente qualquer subarray de tamanho menor que $hi - lo + 1$. Após **PARTITION**, o pivô está em sua posição final definitiva e as duas partições $A[lo..p-1]$ e $A[p+1..hi]$ são estritamente menores que o array original — pela hipótese de indução, as

chamadas recursivas as ordenam corretamente. Como todo elemento à esquerda do pivô vem antes dele e todo elemento à direita não vem antes dele, o array completo fica ordenado.

7.1.2 Heapsort

SIFT-DOWN (também conhecida como *heapify*, é a indução na altura da subárvore enraizada em raiz):

Caso base: raiz é folha (filho > fim) — subárvore de um elemento é trivialmente um heap válido.

Passo indutivo: assume-se que as subárvores dos filhos de raiz já são heaps válidos (garantido por **BUILD-MAX-HEAP** processar de baixo para cima, ou pela iteração anterior de **HEAPSORT**).

SIFT-DOWN escolhe o maior entre raiz e seus filhos; se um filho for maior, troca e continua o processo na nova posição — isso preserva heap nas subárvores não afetadas e, ao terminar (quando raiz não é menor que seus filhos ou é folha), restaura a propriedade de heap em toda a subárvore.

BUILD-MAX-HEAP (invariante): antes de processar o índice i , todas as subárvores enraizadas em índices $> i$ já são heaps válidos (pois nós acima de $n/2$ são folhas e nós são processados em ordem decrescente, ou seja, sempre de baixo para cima). **SIFT-DOWN**(i) então restaura a propriedade de heap em i assumindo filhos já “heapificados” — pela indução acima, correto. Ao final ($i=0$), o array inteiro é um max-heap.

HEAPSORT (invariante de laço): no início de cada iteração, $A[0..fim]$ é um max-heap e $A[fim+1..n-1]$ contém os $n-fim-1$ maiores elementos, já em posição final e ordenados. A raiz do heap ($A[0]$) é, por definição de max-heap, o elemento que não vem antes de nenhum outro em $A[0..fim]$ — trocá-lo para $A[fim]$ coloca o próximo maior elemento em sua posição definitiva; **SIFT-DOWN** restaura o heap no prefixo restante. Ao terminar ($fim=0$), o array está totalmente ordenado.

7.1.3 MergeSort

MERGE-SORT:

Caso base: $hi-lo \leq 1$, 0 ou 1 elemento, trivialmente ordenado.

Passo indutivo: assumo que **MERGE-SORT** ordena corretamente qualquer subarray estritamente menor. As chamadas recursiva (linhas 3 e 4) ordenam $A[lo..meio-1]$ e $A[meio..hi-1]$. **MERGE** combina as sequência em uma só $\Rightarrow A[lo..hi-1]$ fica ordenado.

MERGE: no início de cada iteração, $b[lo..k-1]$ contém os $k-lo$ menores elementos dentre os já “consumidos” de $A[lo..meio-1]$ e $A[meio..hi-1]$, enquanto i e j apontam para os próximos elementos não “consumidos” em cada metade.

Inicialmente, $k=lo$ e $B[lo..lo-1]$ é vazio.

Progressivamente, como ambas as metades já chegam ordenadas (conforme hipótese de indução do **MERGE-SORT** acima), comparar $A[i]$ com $A[j]$ e copiar o menor deles para $B[k]$ preserva a ordem global, isto é, qualquer elemento que ainda não foi copiado em qualquer metade é maior ou igual ao que acabou de ser copiado.

Finalmente, quando um dos ponteiros exaure a sua metade, o resto da outra já está ordenado e é maior que todos já copiados. O array final $B[lo..hi-1]$ é a intercalação ordenada das metades.

8. Análise no modelo RAM

8.1.1 Quicksort

PARTITION sobre um subarray de tamanho m faz $m-1$ comparações — custo $\Theta(m)$. O custo total de **QUICKSORT** é dado pela soma dos custos de partição em cada nível de recursão, que depende de quão balanceadas são as partições.

8.1.2 Heapsort

SIFT-DOWN sobre uma subárvore de altura h custa $O(h)$, pois desce no máximo um nível por iteração e a árvore binária tem altura $\lfloor \log_2 n \rfloor$.

BUILD-MAX-HEAP chama **SIFT-DOWN** uma vez por nó interno — apesar de parecer $O(n \log n)$ à primeira vista, a soma ponderada pelas alturas reais (a maioria dos nós está perto das folhas, com altura pequena) resulta em $\Theta(n)$.

A fase de extração (laço principal de **HEAPSORT**) executa $n-1$ chamadas de **SIFT-DOWN**, cada uma custando $O(\log n) \Rightarrow \Theta(n \log n)$.

Total: $\Theta(n) + \Theta(n \log n) = \Theta(n \log n)$.

8.1.3 Mergesort

MERGE sobre duas metades de tamanho total m custa $\Theta(m)$: cada elemento é comparado (no máximo uma vez por posição do array de saída) e copiado exatamente uma vez. A recursão divide o problema ao meio em cada chamada, resultando em $\lceil \log_2 n \rceil$ níveis de recursão, cada nível somando custo $\Theta(n)$ (a soma dos tamanhos dos merges em cada nível é sempre n).

9. Melhor, pior e caso médio

9.1.1 Quicksort

Melhor caso — $\Omega(n \log n)$: o pivô sempre divide o subarray em duas metades aproximadamente iguais (ex.: pivô é sempre a mediana).

Pior caso — $O(n^2)$: o pivô é sempre o menor ou o maior elemento da partição (ocorre, por exemplo, com array já ordenado, já que o pivô é $A[hi]$), gerando partições de tamanho 0 e $n-1$ em cada chamada.

Caso médio — $\Theta(n \log n)$: para uma permutação aleatória da entrada, o pivô cai, em expectativa, em posições suficientemente variadas para que a recursão se comporte como $O(n \log n)$; a prova formal usa o valor esperado do número de comparações via indicadoras (cada par de elementos é comparado no máximo uma vez, e a probabilidade de comparação decai com a distância entre eles no array ordenado final).

9.1.2 Heapsort

Heapsort é notável por ter **$\Theta(n \log n)$ no melhor, pior e caso médio** — a estrutura de heap (árvore quase completa) garante altura $O(\log n)$ independentemente de como os dados de entrada estão dispostos; o que muda entre os casos são apenas constantes (número de trocas durante o **SIFT-DOWN**), não a ordem assintótica.

9.1.3 Mergesort

Mergesort é, como o heapsort, **$\Theta(n \log n)$ no melhor, pior e caso médio** — a divisão é sempre ao meio (não depende dos valores dos dados) e o **MERGE** sempre custa **$\Theta(m)$** para intercalar m elementos, não importa a ordem de entrada. A única variação entre os casos é o número exato de comparações dentro de cada **MERGE** (pode ser menor se um lado se esgota mais cedo), mas isso afeta apenas a constante, não a ordem assintótica.

9.1.4 Busca Indexada

Considerando:

N = número de documentos;

Q = número de termos da consulta;

P = número total de ocorrências nos *postings lists* dos termos da consulta;

C = número de documentos candidatos distintos;

K = quantidade de resultados desejados;

V = número de termos distintos do vocabulário;

P_{total} = número total de pares (*termo, documento*) no índice;

Temos:

- **Complexidade Temporal:** No melhor caso, a busca não encontraria nenhum documento correspondente aos termos passados, deste modo, sendo necessário apenas percorrer o texto da consulta para normalizá-la, sendo assim, no melhor caso sua complexidade seria $\Omega(Q)$. No caso médio, sua complexidade é $\Theta(P+CQ+CK)$, pois o algoritmo extrai os endereços dos documentos

candidatos no índice invertido pré-processado a partir dos termos da consulta e após isso realiza uma busca linear apenas nos documentos candidatos. Levando em consideração que K pode ser um valor relativamente irrelevante, podemos simplificar o caso médio para $\Theta(P+CQ)$. No pior caso, quando os termos consultados aparecem em praticamente todos os documentos, $C \approx N$ e $P = O(NQ)$, resultando em $O(NQ+NK)$, ou aproximadamente $O(NQ)$.

- **Complexidade Espacial:** O índice invertido requer $O(V+P_{total})$ espaço, sendo V o número de termos distintos e P_{total} o número de associações termo-documento. Durante a consulta, são necessários $O(C+K)$ espaços adicionais, que no pior caso correspondem a $O(N)$.

10. Recorrências

10.1.1 Quicksort

Melhor caso: $T(n) = 2T(n/2) + \Theta(n)$.

Pelo Teorema Mestre ($a=2$, $b=2$, $f(n)=\Theta(n) = \Theta(n^{\log_b a})$) $\Rightarrow T(n) = \Theta(n \log n)$.

Pior caso: $T(n) = T(n-1) + \Theta(n)$. Por expansão: $T(n) = \Theta(n) + \Theta(n-1) + \dots + \Theta(1) = \Theta(n^2)$.

Caso médio: $T(n) = (1/n) \cdot \sum_{q=0}^{n-1} [T(q) + T(n-1-q)] + \Theta(n)$, cuja solução (por indução) também resulta em $\Theta(n \log n)$.

10.1.2 Heapsort

SIFT-DOWN em função da altura h : $T(h) = T(h-1) + O(1) \Rightarrow T(h) = O(h) = O(\log n)$.

BUILD-MAX-HEAP: não é uma recorrência simples, mas um somatório — $T(n) = \sum_{h=0}^{\log n} (n/2^{(h+1)}) \cdot O(h) = O(n \cdot \sum_{h=0}^{\infty} h/2^h) = O(n)$, já que $\sum h/2^h$ converge para uma constante.

10.1.3 Mergesort

$T(n) = 2T(n/2) + \Theta(n)$ (duas chamadas recursivas sobre metades do problema custo linear do merge).

Pelo Teorema Mestre, temos $a=2$, $b=2$ e $f(n)=\Theta(n)=\Theta(n^{\log_b a})$

$\Rightarrow T(n) = \Theta(n \log n)$.

Alternativamente, por árvore de recursão: $\log_2 n$ níveis, cada nível custando $\Theta(n)$ no total (soma dos merges naquele nível) $\Rightarrow T(n) = \Theta(n) \cdot \Theta(\log n) = \Theta(n \log n)$.

11. Metodologia experimental

A avaliação experimental foi planejada com o objetivo de analisar o comportamento das diferentes estratégias de recuperação e seleção implementadas no protótipo, considerando aspectos de eficiência computacional, escalabilidade e qualidade dos resultados recuperados.

O experimento foi estruturado de forma automatizada por meio de um programa de benchmark que executa combinações de configurações algorítmicas, consultas, tamanhos de corpus e valores de k . Para cada execução são coletadas métricas relacionadas ao tempo, memória, número de comparações e qualidade da recuperação.

O benchmark compila previamente o mecanismo de busca em um executável e, em seguida, realiza as execuções experimentais utilizando esse binário, evitando que o tempo de compilação seja incluído repetidamente nas medições de cada cenário.

11.1 Ambiente de execução

Os experimentos foram realizados em um computador com sistema operacional Windows 11, utilizando o Windows Subsystem for Linux (WSL) como ambiente de execução. O equipamento utilizado possui as seguintes características: processador Intel(R) Core(TM) i7-8550U CPU @ 1.80 GHz; 8 GB de memória RAM; arquitetura amd64; linguagem Go 1.26.0.

As implementações analisadas são executadas em CPU e não utilizam aceleração por GPU. Dessa forma, as placas gráficas presentes no dispositivo não participam diretamente do processamento dos algoritmos avaliados.

O benchmark registra automaticamente no arquivo de resultados informações sobre sistema operacional, arquitetura, versão do Go, processador, quantidade de núcleos e memória RAM da máquina utilizada.

11.2 Corpus e pré-processamento

O corpus corresponde à documentação pública da GitHub REST API, obtida a partir de sua especificação OpenAPI 3.0.3.

A etapa de ingestão transforma a especificação original em um arquivo JSONL no qual cada par composto por método HTTP e caminho da API corresponde a um documento. Durante esse processo são realizadas operações de extração, limpeza e normalização textual.

O corpus completo contém 1.224 documentos. O mecanismo de busca calcula informações como frequência de termos por campo, frequência de documentos e IDF, utilizadas posteriormente no cálculo de relevância.

Para analisar o comportamento dos algoritmos em diferentes tamanhos de entrada, o benchmark utiliza tanto o corpus completo, com 1.224 documentos, quanto um subconjunto correspondente à metade do corpus, contendo 612 documentos.

11.3 Consultas e gabarito de relevância

A qualidade da recuperação é avaliada utilizando um gabarito contendo dez perguntas. Para cada consulta são definidos previamente um ou mais identificadores de operações da API considerados relevantes.

Entre as consultas utilizadas estão:

- criação de repositório em uma organização;
- listagem de alertas do Dependabot;
- criação ou atualização de segredo de repositório;
- exclusão de webhook;
- listagem de branches;
- criação de equipe;
- criação de issue;
- criação de release;
- merge de pull request;
- cancelamento de execução de workflow.

O gabarito contém dez consultas e seus respectivos operationId esperados. Em alguns casos, mais de uma operação é considerada relevante, como nas consultas relacionadas a merge de pull request e cancelamento de workflow. Essa abordagem permite avaliar não apenas o tempo dos algoritmos, mas também se os documentos corretos são recuperados e em qual posição aparecem no ranking.

11.4 Configurações principais

O protótipo disponibiliza três configurações principais, representadas no programa por meio de presets.

Preset 1 — busca linear com seleção Top-k

O primeiro cenário utiliza busca linear sobre todos os documentos do corpus. Os documentos são pontuados e os k melhores resultados são mantidos por meio de uma estratégia de inserção ordenada Top-k.

A complexidade da seleção é da ordem de $\Theta(nk)$.

Preset 2 — busca indexada com seleção Top-k

O segundo cenário utiliza um índice invertido que associa cada termo aos documentos em que ele ocorre. Dessa forma, somente os documentos contendo pelo menos um termo da consulta são considerados candidatos à pontuação.

A seleção dos k melhores resultados utiliza a mesma estratégia Top-k da primeira configuração, possibilitando comparar diretamente a busca linear com a busca indexada.

Preset 3 — busca linear com ordenação completa utilizando MergeSort

O terceiro cenário utiliza busca linear para pontuar os documentos e, posteriormente, ordena todos os candidatos utilizando MergeSort antes de selecionar os k primeiros resultados.

A seleção baseada em ordenação completa possui complexidade $\Theta(n \log n)$.

Os três presets utilizam a mesma função de pontuação e o mesmo critério de comparação de relevância. Dessa maneira, para uma mesma consulta, espera-se que produzam a mesma ordenação lógica dos resultados, diferindo principalmente no custo computacional de recuperação e seleção.

11.5 Estratégias adicionais de seleção e ordenação

Além dos três presets principais, o benchmark realiza experimentos adicionais com diferentes formas de selecionar os k melhores documentos.

São avaliadas as seguintes estratégias:

- topk: inserção ordenada durante a varredura;
- heap: manutenção de um min-heap de tamanho k ;
- sort: ordenação completa dos candidatos seguida do corte dos k primeiros.

Quando a estratégia sort é utilizada, podem ser selecionados diferentes algoritmos de ordenação:

- QuickSort;
- HeapSort;
- MergeSort;
- `slices.SortFunc`, da biblioteca padrão do Go, utilizado como baseline de referência.

O QuickSort implementado utiliza pivô determinado pela mediana de três; o HeapSort é executado in-place; e o MergeSort utiliza um buffer auxiliar proporcional ao número de elementos.

11.6 Matriz experimental

O benchmark organiza os experimentos em uma matriz de cenários.

Para a comparação dos três presets principais são utilizados:

- corpus completo, com 1.224 documentos;
- metade do corpus, com 612 documentos;
- $k = 5$.

Além desses cenários, são realizados experimentos específicos para avaliar o impacto do valor de k e da estratégia de seleção. Nessas execuções são considerados $k \in \{5, 50, 500, 1224\}$ e as estratégias Top-k, heap, ordenação com MergeSort, ordenação com QuickSort e ordenação com a implementação padrão da linguagem Go.

No total, o benchmark atual possui 27 cenários. Cada cenário é executado para as dez consultas do gabarito e cada combinação é repetida duas vezes. Assim, o número total de execuções é $27 \times 10 \times 2 = 540$ execuções experimentais.

O programa percorre explicitamente cada combinação de cenário, consulta e repetição, armazenando os resultados obtidos em arquivos destinados à análise posterior.

11.7 Métricas de desempenho

Para cada execução são registradas métricas destinadas a caracterizar o custo computacional do método utilizado.

As principais métricas são:

- N : número de documentos presentes no corpus;
- quantidade de candidatos com score maior que zero;
- número de comparações realizadas durante a seleção;
- tempo de carregamento do corpus;
- tempo de construção do índice invertido;
- memória de heap utilizada após o carregamento e eventual construção do índice;
- tempo de processamento da consulta;
- valor de k ;
- configuração e estratégia de seleção utilizadas.

A saída de uma consulta apresenta essas informações de forma estruturada, distinguindo especialmente o tempo de carregamento, construção do índice e execução da consulta.

Essa separação permite evitar que o custo inicial de construção do índice seja confundido com o custo de uma consulta indexada propriamente dita.

11.8 Métricas de qualidade da recuperação

Além do desempenho computacional, é avaliada a qualidade dos resultados obtidos utilizando o gabarito de consultas.

Para cada consulta são registrados:

- posição em que uma operação considerada relevante aparece;
- ocorrência ou não de uma resposta relevante entre os k primeiros resultados;
- $\text{Precision@}k$;
- número de consultas sem resultado relevante no Top- k .

A $\text{Precision@}k$ é calculada a partir da quantidade de documentos relevantes presentes entre os k primeiros resultados.

Como a maior parte das consultas do gabarito possui apenas uma operação considerada relevante, o valor máximo de Precision@k nesses casos corresponde a $1/k$. Por esse motivo, também são considerados particularmente importantes o número de acertos no Top-k e a posição média da resposta relevante.

11.9 Procedimento experimental

Antes da execução da matriz de benchmark, o mecanismo de busca é compilado uma única vez por meio do comando equivalente a:

```
go build -o bin/search ./searchEngine
```

O benchmark então carrega as dez consultas presentes no gabarito e percorre sistematicamente cada cenário experimental.

Para cada combinação de cenário e consulta são realizadas duas repetições. O executável compilado é iniciado com os parâmetros correspondentes ao cenário, sua saída é capturada e posteriormente processada para obtenção das métricas de desempenho e qualidade.

Os resultados são armazenados em dois formatos:

- `artefatos/benchmark_results.txt`, contendo o log textual das execuções;
- `artefatos/benchmark_results.csv`, contendo os dados estruturados para posterior análise estatística e geração de gráficos.

O arquivo CSV contém, entre outros campos, cenário, preset, configuração, estratégia de seleção, valor de k , tamanho do corpus, consulta, repetição, número de candidatos, número de comparações, tempos, memória, posição da resposta relevante e Precision@k.

11.10 Reprodutibilidade

O procedimento experimental foi estruturado para permitir sua reprodução diretamente a partir do repositório.

A sequência principal de execução consiste em:

1. executar a pipeline de ingestão;
2. executar os testes automatizados;
3. executar a avaliação das dez consultas do gabarito;
4. executar o benchmark completo.

O repositório documenta os seguintes comandos:

```
go run ./ingestDataPipeline
go test ./...
go run ./searchEngine -eval
go run ./benchmark
```

Os testes automatizados abrangem normalização, comparação de relevância, algoritmos de ordenação, estratégias de seleção, função de score, equivalência entre as buscas linear e indexada e métricas de avaliação.

Essa combinação de código, gabarito, comandos documentados, registros de hardware e arquivos CSV permite que as execuções sejam repetidas e comparadas em outros ambientes computacionais.

12. Resultados e gráficos

Os experimentos foram executados utilizando a matriz definida na metodologia experimental, totalizando 540 execuções distribuídas entre 27 cenários, 10 consultas e duas repetições por combinação. A análise dos resultados foi dividida em quatro aspectos principais: comparação entre as três configurações principais, efeito do tamanho do corpus, impacto do valor de k sobre as estratégias de seleção e qualidade da recuperação.

12.1 Comparação entre as configurações principais

Inicialmente foram comparados os três presets definidos no sistema:

- Preset 1: busca linear com seleção Top-k;
- Preset 2: busca indexada com seleção Top-k;
- Preset 3: busca linear com ordenação completa por MergeSort.

Para $k=5$, foram analisados tanto o corpus completo, com 1.224 documentos, quanto o subconjunto com 612 documentos.

Preset	N	Tempo médio da consulta (μs)	Mediana (μs)	Comparações médias	Candidatos médios
Linear + Top-k	612	440,4	429	284,9	235,6
Indexada + Top-k	612	246,95	179	283,6	235,6
Linear + MergeSort	612	489,05	442,5	1.583,60	235,6
Linear + Top-k	1.224	942,95	885	585,3	519,3
Indexada + Top-k	1.224	665,25	333	577,8	519,3
Linear + MergeSort	1.224	982,85	915	4.077,30	519,3

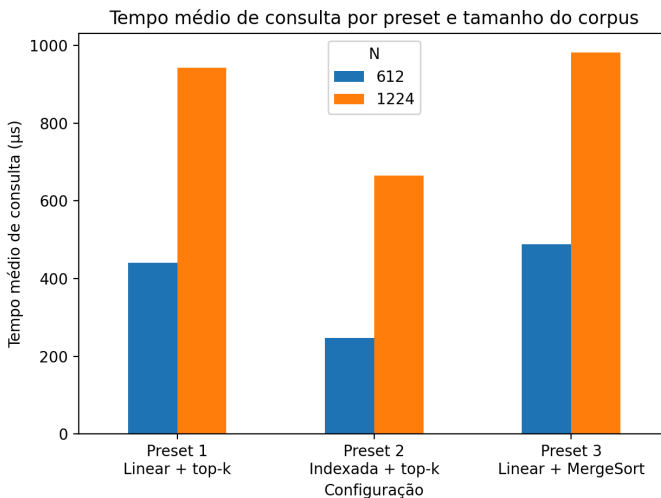
No corpus completo, a busca indexada apresentou o menor tempo médio de consulta, com aproximadamente 665,25 μs. A busca linear com Top-k apresentou média de 942,95 μs, enquanto a configuração com MergeSort apresentou 982,85 μs.

Comparando diretamente os dois primeiros presets, a busca indexada apresentou redução de aproximadamente 29,5% no tempo médio de consulta em relação à busca linear no corpus completo.

Esse comportamento também apareceu no corpus de 612 documentos. A busca indexada apresentou tempo médio de 246,95 μ s, contra 440,40 μ s da busca linear, correspondendo a uma redução aproximada de 43,9%.

A Figura 1 apresenta essa comparação.

Figura 1 - Tempo médio de consulta por configuração e tamanho do corpus.



12.2 Número de comparações

Além do tempo de execução, foi avaliado o número de comparações realizadas durante a etapa de seleção dos resultados.

Nos presets 1 e 2, que utilizam Top-k, o número de comparações foi semelhante. Para o corpus completo, foram registradas em média 585,3 comparações para a busca linear e 577,8 para a busca indexada.

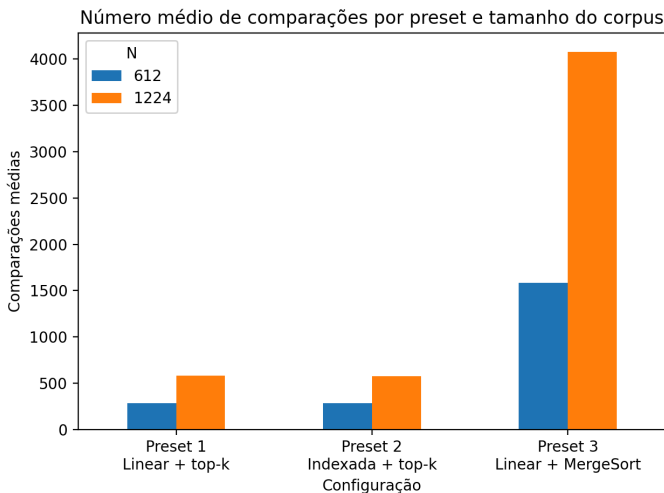
A principal diferença apareceu no Preset 3, baseado em ordenação completa por MergeSort. Nesse caso, foram realizadas em média 4.077,3 comparações para o corpus completo, valor significativamente superior ao observado nas configurações Top-k.

Para o corpus reduzido, a mesma tendência foi observada: o MergeSort realizou aproximadamente 1.583,6 comparações, enquanto as configurações Top-k permaneceram próximas de 285 comparações.

Esses resultados mostram que, para $k=5$, a estratégia de manter apenas os melhores resultados durante a varredura evita grande parte do custo associado à ordenação completa de todos os candidatos.

A Figura 2 apresenta o número médio de comparações realizado pelos três presets.

Figura 2 - Número médio de comparações por configuração.



12.3 Custo da indexação e uso de memória

A busca indexada possui um custo adicional que não ocorre na busca linear: a construção do índice invertido.

Para o corpus completo, o tempo médio de construção do índice foi de aproximadamente 11.916 μ s, ou 11,9 ms. Para 612 documentos, esse valor foi de aproximadamente 5.977 μ s, ou 6,0 ms.

Portanto, apesar de a consulta indexada ser mais rápida após a construção do índice, existe um custo inicial de preparação. Esse resultado evidencia a diferença entre aplicações em que o índice é construído uma única vez e reutilizado em várias consultas e situações em que o índice precisaria ser reconstruído frequentemente.

Também foi observado aumento no consumo de memória com a utilização do índice invertido. Para o corpus completo, a configuração linear utilizou em média aproximadamente 6.542 kb de heap, enquanto a indexada utilizou aproximadamente 7.257 kb, um aumento de cerca de 10,9%.

Esse aumento é esperado devido à necessidade de armazenar a estrutura adicional que associa termos aos documentos correspondentes.

12.4 Efeito do tamanho do corpus

A redução do corpus de 1.224 para 612 documentos provocou redução no tempo médio de consulta em todas as três configurações.

Na busca linear com Top-k, o tempo médio caiu de 942,95 μ s para 440,40 μ s. Na busca indexada, passou de 665,25 μ s para 246,95 μ s. No MergeSort, caiu de 982,85 μ s para 489,05 μ s.

Também houve redução no número de candidatos e de comparações, indicando que o custo computacional acompanha o crescimento do tamanho da entrada.

Contudo, a redução do corpus também afetou a qualidade da recuperação. No corpus completo, as configurações obtiveram acerto no Top-5 em 9 das 10 consultas. No subconjunto com 612 documentos, apenas 4 das 10 consultas apresentaram uma resposta considerada relevante entre os cinco primeiros resultados.

Essa diferença não deve ser interpretada apenas como perda de qualidade do algoritmo. Como o subconjunto é formado pelos primeiros 612 documentos do corpus, algumas operações presentes no gabarito deixam de estar disponíveis na entrada reduzida. Por exemplo, algumas consultas executadas sobre o subconjunto não contêm o endpoint esperado entre os documentos considerados.

Assim, os resultados com $N=612$ são particularmente adequados para análise de desempenho e escalabilidade, mas devem ser interpretados com cautela quando utilizados para avaliação da qualidade da recuperação.

12.5 Impacto do valor de k

Também foram analisadas diferentes estratégias de seleção dos melhores resultados para:

$$k \in \{5, 50, 500, 1224\}$$

Foram comparadas as estratégias Top- k , heap, MergeSort, QuickSort e a função de ordenação da biblioteca padrão do Go.

O comportamento mais evidente ocorreu no número de comparações da estratégia Top- k .

Estratégia	$k=5$	$k=50$	$k=500$	$k=1224$
Top- k	585	4.001	61.841	86.667
Heap	596	1.884	6.856	8.281
MergeSort	4.077	4.077	4.077	4.077
QuickSort	5.122	5.122	5.122	5.122
Std	4.996	4.996	4.996	4.996

Para valores pequenos de k , a seleção Top- k realizou poucas comparações e apresentou comportamento competitivo. Entretanto, à medida que k cresceu, o número de comparações aumentou rapidamente.

Com $k=500$, a estratégia Top- k realizou mais de 61 mil comparações em média. Para $k=1224$, esse número ultrapassou 86 mil.

Por outro lado, as estratégias baseadas em ordenação completa mantiveram aproximadamente o mesmo número de comparações independentemente do valor de k , pois todos os candidatos são ordenados antes do corte dos resultados.

O heap apresentou comportamento intermediário, aumentando o número de comparações conforme k cresce, porém em proporção muito inferior à observada no Top- k .

Esse resultado evidencia que a escolha da estratégia de seleção depende fortemente do número de resultados desejados.

A Figura 3 apresenta o efeito de k sobre o tempo médio de consulta, enquanto a Figura 4 apresenta seu impacto sobre o número de comparações.

Figura 3 - Tempo médio de consulta em função de k .

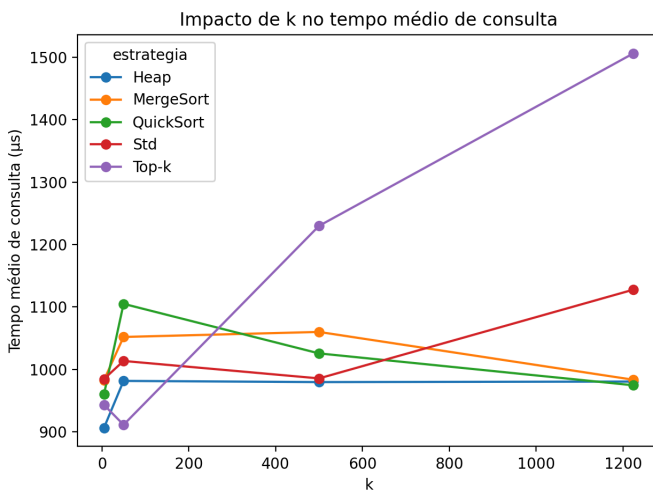
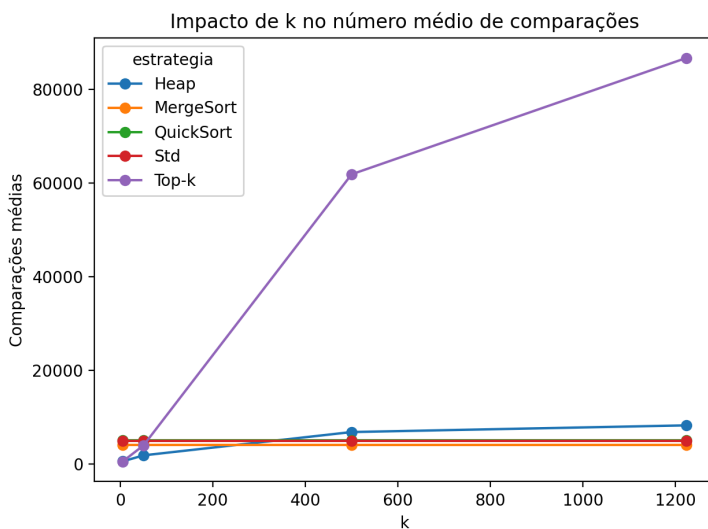


Figura 4 - Número médio de comparações em função de k .



12.6 Qualidade da recuperação

A avaliação executada com $k=5$, utilizando o corpus completo e a configuração linear com Top-k, apresentou os seguintes resultados:

- 10 consultas avaliadas;
- 9 consultas com ao menos uma resposta relevante no Top-5;
- nenhuma consulta sem resultados;
- Precision@5 média de 0,220;
- posição média das respostas relevantes igual a 1,6;
- tempo médio de consulta de aproximadamente 859 μ s.

A única consulta em que o endpoint esperado não apareceu entre os cinco primeiros resultados foi “create an issue”. Nesse caso, operações relacionadas a campos e tipos de issue receberam pontuação superior à operação genérica de criação de uma issue.

Em contraste, várias consultas recuperaram o endpoint esperado na primeira posição, entre elas:

- create a repository in an organization;
- delete a repository webhook;
- list branches;
- create a team;
- merge a pull request;
- cancel a workflow run.

Para a consulta relacionada à criação de repositório, por exemplo, a operação `repos/create-in-org` foi classificada em primeiro lugar.

Na consulta sobre alertas do Dependabot, o endpoint esperado para repositórios apareceu na terceira posição, atrás das operações equivalentes para enterprise e organização.

Esses resultados demonstram que o mecanismo de recuperação lexical apresenta bom comportamento para a maior parte das consultas do gabarito, embora ainda existam ambiguidades semânticas em consultas curtas ou genéricas, como ‘create an issue’.

12.7 Síntese dos resultados

De forma geral, os experimentos mostraram que:

1. a busca indexada apresentou menor tempo médio de consulta que a busca linear nos tamanhos de corpus avaliados;
2. essa melhoria possui como contrapartida o custo de construção do índice e maior utilização de memória;
3. para valores pequenos de k , a seleção Top-k apresenta baixo número de comparações;
4. para valores grandes de k , seu custo cresce significativamente;
5. o heap apresenta crescimento mais controlado conforme k aumenta;

6. as estratégias de ordenação completa mantêm custo praticamente independente de k ;
7. no corpus completo, o sistema recuperou ao menos uma resposta relevante no Top-5 em 9 das 10 consultas avaliadas;
8. A redução artificial do corpus compromete a interpretação das métricas de qualidade quando o documento esperado deixa de fazer parte da entrada.

Esses resultados servem de base para a discussão de escalabilidade apresentada na seção seguinte.

13. Discussão de escalabilidade

A análise de escalabilidade foi realizada considerando principalmente dois parâmetros: o tamanho do corpus, representado por N , e a quantidade de resultados solicitados, representada por k . Os experimentos permitem observar como as diferentes estratégias de busca e seleção respondem ao crescimento desses parâmetros e em que condições cada abordagem tende a se tornar mais ou menos adequada.

13.1 Efeito do crescimento do corpus

Os experimentos com 612 e 1.224 documentos mostram aumento consistente no custo das três configurações principais quando o tamanho do corpus cresce.

Na busca linear com seleção Top- k , o tempo médio de consulta aumentou de aproximadamente 440,40 μ s para 942,95 μ s quando o corpus passou de 612 para 1.224 documentos. Esse comportamento é compatível com a necessidade de percorrer os documentos do corpus e calcular a relevância dos candidatos.

A configuração baseada em MergeSort também apresentou crescimento semelhante, passando de aproximadamente 489,05 μ s para 982,85 μ s. Entretanto, essa abordagem apresentou um número de comparações significativamente maior, pois após a pontuação dos candidatos é necessário ordenar o conjunto antes de selecionar os k primeiros.

A busca indexada apresentou tempos médios de aproximadamente 246,95 μ s para 612 documentos e 665,25 μ s para 1.224 documentos. Apesar do aumento observado, ela permaneceu abaixo das configurações lineares nos dois tamanhos avaliados.

Esse resultado pode ser explicado pelo uso do índice invertido. Enquanto a busca linear percorre todos os documentos, a busca indexada pode restringir a etapa de pontuação aos documentos que possuem ao menos um termo presente na consulta. O mecanismo indexado foi implementado exatamente com esse objetivo: recuperar candidatos por meio das listas de ocorrência dos termos e pontuar apenas esse subconjunto.

Entretanto, o ganho proporcionado pelo índice depende da seletividade da consulta. Consultas formadas por termos frequentes podem produzir um grande número de candidatos. Na consulta ‘create a repository in an organization’, por exemplo, 1.022 dos 1.224 documentos foram considerados candidatos. Nesse tipo de situação, o índice elimina apenas uma pequena parcela do corpus e sua vantagem tende a ser menor.

Por outro lado, consultas mais específicas podem produzir conjuntos de candidatos consideravelmente menores. Na consulta ‘cancel a workflow run’, apenas 94 documentos foram considerados candidatos no corpus completo. Nesses casos, a indexação possui maior potencial para evitar trabalho desnecessário.

13.2 Custo de construção do índice

A maior eficiência observada na busca indexada durante as consultas possui como contrapartida um custo inicial de construção do índice invertido.

Nos experimentos realizados com o corpus completo, a construção do índice demandou aproximadamente 11,9 ms em média. Esse custo é superior ao tempo de uma consulta individual, que permaneceu na ordem de centenas de microssegundos.

Isso significa que o uso da busca indexada é especialmente vantajoso em cenários em que o índice pode ser construído uma única vez e reutilizado para múltiplas consultas.

Considere, de forma simplificada:

$$T_{indexado} = T_{construção} + q \cdot T_{consulta indexada}$$

enquanto a busca linear apresenta

$$T_{linear} = q \cdot T_{consulta linear}$$

onde q representa o número de consultas executadas sobre o mesmo corpus.

Para poucas consultas, o custo de criação do índice pode superar o ganho obtido em cada busca individual. À medida que o número de consultas cresce, o custo inicial é amortizado e a busca indexada tende a se tornar mais vantajosa.

Esse cenário é particularmente compatível com aplicações de recuperação de contexto, nas quais um mesmo conjunto documental costuma atender múltiplas consultas sucessivas.

13.3 Impacto do valor de k

Os experimentos também demonstraram que o parâmetro k possui forte influência sobre algumas estratégias de seleção.

A estratégia Top- k apresentou aproximadamente 585 comparações, em média, para $k=5$. Quando k aumentou para 50, esse número passou para cerca de 4.001 comparações. Para $k=500$, foram registradas aproximadamente 61.841 comparações, e para $k=1224$, mais de 86 mil comparações.

Esse crescimento está relacionado à forma como a estratégia mantém uma lista ordenada dos melhores resultados encontrados durante a varredura. Quanto maior o valor de k , maior o custo necessário para inserir novos candidatos na estrutura de resultados.

Esse comportamento é compatível com a complexidade $\Theta(nk)$ associada à estratégia Top-k implementada no projeto.

Para valores pequenos de k , como $k=5$, essa abordagem é eficiente, pois evita ordenar todos os candidatos. Entretanto, quando k se aproxima de n , sua vantagem desaparece e seu custo cresce substancialmente.

13.4 Escalabilidade da estratégia baseada em heap

A estratégia baseada em heap apresentou comportamento intermediário.

Seu número médio de comparações aumentou de aproximadamente 596 para $k=5$ para cerca de 8.281 em $k=1224$. Embora exista crescimento com o aumento de k , ele foi muito inferior ao observado no Top-k.

Esse comportamento está de acordo com a complexidade $\Theta(n \log k)$ da manutenção de um heap de tamanho k .

Dessa forma, a estratégia de heap tende a ser especialmente interessante quando k não é suficientemente pequeno para tornar o Top-k barato, mas ainda é menor do que o número total de candidatos.

Ela representa, portanto, uma solução intermediária entre manter uma pequena lista ordenada de resultados e ordenar completamente todo o conjunto.

13.5 Ordenação completa

As estratégias baseadas em ordenação completa apresentaram comportamento diferente em relação a k .

MergeSort, QuickSort e a ordenação da biblioteca padrão mantiveram praticamente o mesmo número de comparações para os diferentes valores de k . Isso ocorre porque a lista completa de candidatos é ordenada antes de serem selecionados os k primeiros resultados.

Assim, o custo principal é determinado pelo número de candidatos, e não diretamente pelo número de resultados solicitados.

Para o corpus completo, foram observadas aproximadamente:

- 4.077 comparações para MergeSort;
- 5.122 comparações para QuickSort;
- 4.996 comparações para a implementação padrão da linguagem.

Esse comportamento corresponde à complexidade esperada de algoritmos de ordenação na ordem de $\Theta(n \log n)$ no caso médio das estratégias avaliadas.

A ordenação completa pode apresentar custo desnecessário quando apenas poucos elementos são solicitados. Entretanto, à medida que k se aproxima de n , seu custo passa a ser competitivo em relação a estratégias cujo custo depende diretamente de k .

Os resultados experimentais evidenciam esse ponto: enquanto o Top-k apresentou poucas comparações para $k=5$, ele ultrapassou 86 mil comparações quando $k=1224$, valor muito superior às aproximadamente 4 a 5 mil comparações das estratégias de ordenação completa.

13.6 Relação entre n e k

Os resultados demonstram que não existe uma única estratégia que seja ideal para todos os valores de n e k . De forma geral, o comportamento observado pode ser resumido da seguinte maneira:

Situação	Estratégia com comportamento favorável
k muito pequeno	Top-k
k intermediário	Heap
k próximo de n	Ordenação completa
Muitas consultas sobre o mesmo corpus	Busca indexada
Poucas consultas e corpus pequeno	Busca linear pode permanecer competitiva

Assim, a escolha do algoritmo depende das características da aplicação.

Em um sistema de recuperação de contexto para IA generativa, valores de k costumam ser relativamente pequenos em comparação com o corpus total, pois apenas alguns documentos são normalmente inseridos no contexto enviado ao modelo. Nesse tipo de cenário, estratégias como Top-k ou heap podem evitar o custo de ordenar todos os candidatos.

Por outro lado, à medida que o corpus cresce para dezenas ou centenas de milhares de documentos, a busca linear tende a exigir cada vez mais processamento, tornando estruturas de indexação progressivamente mais relevantes.

13.7 Memória e escalabilidade espacial

A escalabilidade também deve considerar o consumo de memória.

Nos experimentos com o corpus completo, a configuração linear utilizou aproximadamente 6,5 MB de heap, enquanto a busca indexada utilizou aproximadamente 7,3 MB. O aumento está associado principalmente à estrutura do índice invertido.

O mecanismo registra a memória após o carregamento do corpus e a eventual construção do índice.

Assim, existe um trade-off entre tempo e espaço:

- a busca linear exige menos memória adicional, mas precisa percorrer mais documentos;
- a busca indexada utiliza memória adicional para armazenar as listas de ocorrências, porém pode reduzir o conjunto de candidatos consultados.

Para o corpus atual, esse acréscimo de memória é relativamente pequeno. Entretanto, em corpus muito maiores, o tamanho do índice pode se tornar relevante e deverá ser considerado no projeto do sistema.

13.8 Limitações da extrapolação

Embora os resultados indiquem tendências consistentes, o corpus avaliado possui apenas 1.224 documentos. Portanto, os experimentos permitem observar o comportamento relativo das estratégias, mas não são suficientes para determinar com precisão seu desempenho em corpus com milhões de documentos.

Além disso, os tempos de consulta estão na ordem de microssegundos e milissegundos, tornando as medições suscetíveis a efeitos do ambiente de execução, como escalonamento do sistema operacional, uso de cache e outros processos ativos.

Outro aspecto importante é que o número de candidatos depende da consulta. Dessa forma, duas consultas executadas sobre o mesmo corpus podem apresentar custos diferentes mesmo utilizando exatamente a mesma estratégia.

Por esse motivo, os resultados devem ser interpretados como evidência experimental do comportamento dos algoritmos dentro das condições avaliadas, e não como uma garantia de desempenho para qualquer escala ou distribuição de consultas.

13.9 Síntese da escalabilidade

Os experimentos demonstraram três tendências principais.

Primeiramente, o crescimento do corpus aumenta o custo de todas as configurações, porém a utilização de índice invertido reduz o custo das consultas quando o conjunto de candidatos é significativamente menor que o corpus.

Em segundo lugar, o parâmetro k altera substancialmente o comportamento das estratégias de seleção. O Top- k é particularmente eficiente para valores pequenos de k , enquanto seu custo cresce rapidamente à medida que k se aproxima de n . A utilização de heap apresenta crescimento mais moderado, enquanto a ordenação completa possui custo praticamente independente do valor de k .

Por fim, a busca indexada apresenta um trade-off entre custo inicial, memória e tempo de consulta. A construção do índice adiciona uma etapa de pré-processamento, porém esse custo pode ser amortizado quando diversas consultas são executadas sobre o mesmo corpus.

Esses resultados reforçam que a escalabilidade do sistema não depende apenas do número de documentos, mas também do valor de k , da seletividade das consultas, da quantidade de consultas realizadas e das estruturas auxiliares mantidas pelo sistema.

14. Relação com IA generativa e RAG

- Para utilizar os documentos no prompt, seriam inseridas informações como o caminho, o resumo e a URL do conteúdo recuperado, definindo que o modelo utilize apenas as informações presentes no contexto fornecido para formular as respostas.
- Como o valor de k define a quantidade de documentos utilizados pelo modelo, valores altos podem aumentar o custo para utilização, o tempo de processamento e a utilização de documentos menos relevantes na formulação da resposta. Já valores menores tornam o processo mais rápido, mas reduzem a quantidade de informações disponíveis para o modelo. A ordenação também pode impactar na relevância dos documentos com maior pontuação para a formulação das respostas.
- A busca por conteúdo com similaridade lexical permite selecionar documentos que possuem as mesmas palavras presentes no resumo, porém não é capaz de recuperar documentos que apresentam termos relacionados às palavras e que abordam o mesmo tema.
- Como os documentos são recuperados a partir da presença dos termos no resumo, documentos relevantes podem não ser selecionados por não apresentarem as mesmas palavras da consulta. Como também documentos com um número elevado de termos presentes podem ser utilizados, mesmo sendo menos relevantes. Já a consulta utilizando termos genéricos pode implicar na recuperação de documentos que não abordem o mesmo problema.
- O modelo pode gerar informações que não estão presentes nos documentos, mesmo com a instrução de basear as respostas apenas nos documentos recuperados. Para reduzir as chances desse comportamento, pode ser definido que o modelo informe quando não houver informações suficientes para a resposta e também para que apresente referências para as respostas utilizando a documentação.
- Ao definir que o modelo utilize apenas os documentos recuperados como fonte e que as respostas apresentem citações associadas aos documentos, as informações originais podem ser encontradas de forma mais rápida, facilitando a verificação das informações fornecidas pelo modelo.

15. Limitações e ameaças à validade

Antes de falarmos sobre todas as limitações, devemos primeiro constatar as limitações causadas pela escolha da API.

- A primeira limitação é por conta de que as categorias repos e action representam 32% do corpus consultas genéricas tendem a recuperar operações dessas duas categorias.
- Outra foi por conta da frase “The name is not case sensitive”, essa frase aparece em 924 documentos e a frase sobre escopos de tokens OAuth em 456. O que gera um aumento da frequência com que esses termos apareçam.
- 28 documentos não tem *description* e 19 não tem parâmetros nem corpo.

Agora que já falamos sobre as limitações que ocorreram pela escolha da documentação da API, devemos falar sobre as limitações ocorridas pela implementação, execução e testes:

- Tivemos a impossibilidade de executar uma estratégia de chunks que na maioria dos casos sempre é a melhor opção para que o tamanho do documento fique reduzido, ordenado e melhor organizado em geral, porém a documentação utilizada já estava separada em documentos.
- Também tivemos uma limitação quanto a quantidade de algoritmos de ordenação utilizados, como utilizamos apenas três (merge, quick e heap), a pesquisa ficou limitada, quanto mais algoritmos utilizássemos mais dados e com isso mais resultados poderiam ser extraídos.

Além das limitações temos também as ameaças a validade que sempre estão presentes em qualquer pesquisa, no nosso caso acredito que a maior ameaça seja a familiaridade dos integrantes do grupo com a plataforma github, onde todos já trabalharam/utilizaram a plataforma então acabamos tendo vícios de utilização que uma pessoa que nunca utilizou a plataforma acabaria fazendo perguntas que nem pensariamos.

16. Declaração de Uso de IA Generativa

17. Contribuição individual

Ismael Teles Santos

- Inicialização do repo
- Busca linear
- IngestDataPipeline
- Estrutura do corpus

Flavio Emanuel de Oliveira Santos

- Seções 1, 2, 3, 4, 6.1, 7, 8, 9 e 10 do relatório
- PR #05 - [sorting] Implementa estratégias de ordenamento e algumas melhorias
- Criação inicial dos slides e roteiro de apresentação
- Formatação do relatório

Amintas Gomes de Moraes Neto

- Seções 6.2, 9.1.4
- PR #06 - Implementa algoritmo de busca indexada

18. URL do vídeo

19. Referências